

Міністерство освіти і науки України
Національний університет «Львівська політехніка»
Інститут телекомунікацій, радіоелектроніки та електронної техніки
кафедра «Радіоелектронні пристрої та системи»



Звіт з лабораторної роботи №7
з дисципліни «Об'єктно-орієнтоване програмування, частина 2»

Підготував:
ст. групи ІХ-21
Пасько Максим Павлович

Прийняла:
к.т.н., доц. каф. ПАСІ
Плесканка М.В.

Львів 2026р.

Лабораторна робота №7 Реалізація клієнт-серверної взаємодії на основі протоколу WebSocket

Мета: Розробити клієнт-серверну архітектуру, ознайомитися з WebSocket протоколом.

Хід виконання роботи

Розробити WebSocket-клієнт `WebSocketClient`, який дозволяє підключатися до сервера, надсилати повідомлення та отримувати відповіді в асинхронному режимі.

Вимоги до виконання:

- **Реалізувати клас `WebSocketClient`, що містить такі асинхронні методи:**

`connect(url)`: встановлює WebSocket-з'єднання за вказаною адресою.

`send_message(message)`: надсилає повідомлення серверу.

`receive_message()`: отримує повідомлення від сервера.

`close_connection()`: закриває WebSocket-з'єднання.

- **Використовувати реальний WebSocket-сервер для тестування.**
- **Реалізувати логіку обробки помилок, якщо сервер не відповідає або з'єднання перервано.**
- **Надати приклад використання класу `WebSocketClient`, що демонструє відправку та отримання повідомлень в асинхронному режимі.**

```

import asyncio
import websockets

class WebSocketClient: 1 usage

    def __init__(self):
        self.connection = None

    async def connect(self, url): 1 usage
        try:
            self.connection = await websockets.connect(url)
            print("Підключено до сервера:", url)
        except Exception as e:
            print("Помилка підключення:", e)

    async def send_message(self, message): 1 usage
        try:
            if self.connection:
                await self.connection.send(message)
                print("Надіслано:", message)
            else:
                print("З'єднання не встановлено")
        except Exception as e:
            print("Помилка надсилання:", e)

```

```

26  > async def receive_message(self): 1 usage
27  >     try:
28  >         if self.connection:
29             response = await self.connection.recv()
30             print("Отримано:", response)
31         else:
32             print("З'єднання не встановлено")
33     except Exception as e:
34         print("Помилка отримання:", e)
35
36  > async def close_connection(self): 1 usage
37  >     if self.connection:
38         await self.connection.close()
39         print("З'єднання закрито")
40
41  > async def main(): 1 usage
42
43     url = "wss://ws.iffelse.io"
44
45     client = WebSocketClient()
46
47     await client.connect(url)
48     await client.send_message("Hello WebSocket")
49     await client.receive_message()
50
51     await client.close_connection()
52
53     asyncio.run(main())

```

Виведе:

Підключено до сервера: wss://ws.ifelse.io

Надіслано: Hello WebSocket

Отримано: Request served by 6e82931b755587

З'єднання закрито

Контрольні питання до лабораторної роботи

1. Які є основні особливості WebSocket?

WebSocket — це мережевий протокол, який забезпечує двостороннє постійне з'єднання між клієнтом і сервером для швидкого обміну даними в реальному часі.

2. Які особливості WebSocket у порівнянні з HTTP-запитами?

WebSocket має кілька важливих відмінностей від HTTP:

1) Постійне з'єднання

У WebSocket після встановлення з'єднання воно залишається відкритим, і клієнт та сервер можуть обмінюватися повідомленнями багато разів. У HTTP кожен запит створює окреме з'єднання.

2) Двосторонній обмін даними (Full-Duplex)

WebSocket дозволяє клієнту і серверу надсилати повідомлення один одному у будь-який момент. У HTTP зазвичай спочатку клієнт робить запит, а сервер лише відповідає.

3) Менший службовий трафік

У WebSocket передається менше службових заголовків, ніж у HTTP.

3. Як WebSocket-клієнт встановлює з'єднання із сервером у Python?

Для встановлення потрібно використовувати

1) для імпорту import

asyncio

import websockets

2) функція

```
async def connect():
```

3) встановлює з'єднання

```
await websockets.connect("wss://server_address") #ставляєш якісь адрес сервера
```